

291A Neural Information Retrieval Final Report

Video Retrieval

Wesley Truong, Adyah Rastogi

December 2025

1 Objectives and Challenges

The objective of this project is to understand and reproduce the findings from the paper **Queries Are Not Alone: Clustering Text Embeddings for Video Search** [4]. With the rise of social media platforms with predominantly video content, there is a need for improved more advanced video retrieval systems. Traditionally, a text query is searched against the video’s pre-existing metadata (text descriptions of the video, captions if they exist, etc.); metadata often fails to encapsulate the nature of video content, especially since there may be several interpretations and videos may have many topics/themes, not just one main theme. The goal of this paper was to enhance video retrieval by clustering text queries in order to capture the broader semantic meaning of videos.

Although the paper goes into great mathematical and algorithmic detail, many key architectural details and hyperparameters are left ambiguous. Due to the paper’s lack of open-source code transparency, our project aims to recreate the paper’s core architecture while exploring some of these ambiguities.

2 State-of-the-Art Techniques

Previous state-of-the-art techniques like X-Pool [1] introduced a cross-modal attention mechanism where the text sequence attends directly to video frame tokens, producing a text-conditioned video representation used for retrieval. Another paper, TEFAL [2], uses text-conditioned cross-attention blocks where the text query serves the query in attention, and video (and audio) tokens are keys/values, yielding text-conditioned video representations. Both of these methods rely on a single text query (in training, the caption or metadata for a video), and doesn’t build the video’s semantic meaning further. CLIP (Contrastive Language-Image Pre-training) is an AI model from OpenAI that understands the relationship between images and text and provides a text and video encoder that maps embeddings of text and videos to the same latent space [6]. It can perform zero-shot tasks, meaning it can classify an image or match it with a text description without being specifically trained for that particular task. CLIP-based methods like CLIPBERT [3], CLIP4Clip [5], CenterCLIP [9], CLIP-ViP [8] mostly adapt CLIP to video via sparse frame sampling, frame clustering, audio alignment, or prompt tuning.

The state-of-the-art technique in the paper we reference builds upon the techniques in these papers by rethinking queries as an item in a semantic cluster of related texts and uses that cluster to retrieve videos. This expands the search space semantically; if the user queried “cat playing”, the cluster might expand to “kitten playing”, “cat jumping”, “british shorthair jumping on cat tree”, “kitten laser toy”, allowing users to find videos with the same meaning as their query even if the video metadata was never explicitly tagged with those labels. The paper “Queries Are Not Alone: Clustering Text Embeddings for Video Search” was written by Peiyang Liu, Xi Wang, Ziqiang Cui, and Wei Ye and published in ACM SIGIR ’25 [4]. SIGIR specializes in research and development in information retrieval. The researchers are at Peking University and the City University of Hong Kong.

3 Key Algorithms and Techniques

The authors introduce three new techniques: a Text Clusterer, Sweeper module, and VTC-Att (Video-Text Cluster Attention). The goal of these modules is to expand a query into a cluster of semantically similar, related texts to use this cluster as the fundamental retrieval unit. Instead of only aligning a query embedding with video embeddings, it builds a Video-Text Cluster (VTC) representation that captures multiple possible phrasings and nuances of the query, aiming to better model the one-to-many relationship between a short query and diverse relevant videos.

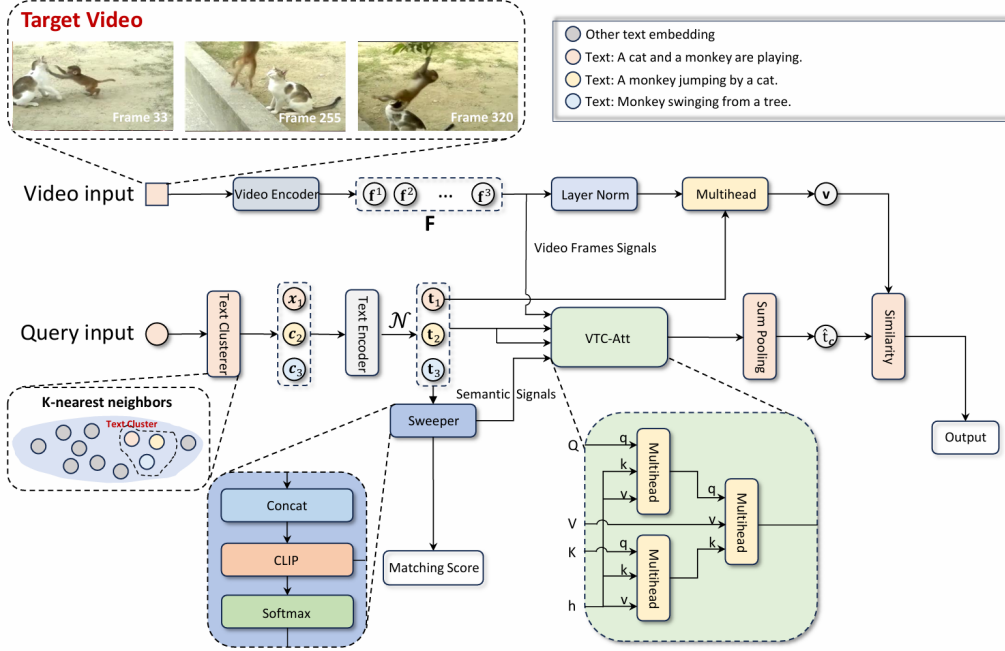


Figure 1: Overview of the end-to-end retrieval algorithm. The same architecture is used for training, but with pairs of text and video as described below.

3.1 Text Clusterer

The **Text Clusterer** is a CLIP-based text encoder trained with a triplet-style contrastive loss and dropout to form a semantically structured embedding space, from which K-nearest text neighbors are used as the query’s cluster, rather than relying on hand-crafted or random clusters. Prior CLIP adaptations did not introduce a dedicated clustering module for retrieval. Then, the CLIP-based text encoder is used to encode the query and texts in the text cluster, and the CLIP video encoder is used to encode the video frames.

3.2 Sweeper

The **Sweeper module** is used to perform noise recognition on the text clusters, identifying and scoring texts based on probability of being noise. It uses cross-attention and takes the query + its cluster and learns to classify cluster members into graded “noise segments” using automatically generated Jaccard-similarity labels; this provides semantic signals s_i indicating which cluster texts are likely noise. The scoring works by placing each text into a “bucket” corresponding to how likely it is noisy

or not (e.g. 4 buckets: bucket 1 (0-25%), bucket 2 (25-50%), etc.). These semantic signals are sent to the next module.

3.3 Video-Text Cluster Attention (VTC-Attention)

The **Video-Text Cluster Attention** (VTC-Attention) module uses multi-head attention that injects the Sweeper’s semantic signals into the query and key, along with the query’s ANN matrix (text-cluster tokens) and video frame tokens to compute a refined text embedding. Through this multi-head attention, the model takes the Sweeper’s signals (flagged texts) and actually decides which cluster texts matter given the specific video. Prior methods use cross-attention between one query and video frames, but this method adds cluster texts as and semantic scores from the Sweeper.

3.4 Joint Training

The text-clusterer is trained separately, and the text encoder, video encoder, sweeper, VTC-Att, and retriever are trained jointly. The retriever computes the similarities between the refined text embedding for a query and video embeddings for all text-video pairs in the batch and applies a contrastive loss (InfoNCE) over positive and in-batch negative pairs. The gradients from this retrieval loss flow through VTC-Att, Sweeper, and the text/video encoders, so the whole stack is optimized jointly for retrieval quality rather than training these modules separately.

At test time, we follow essentially the same pipeline as training, but without updating parameters. For each query, we obtain its cluster via the Text Clusterer and K-nearest neighbors in the text-embedding space. Then, we encode the query, cluster texts, and all candidate videos, run Sweeper + VTC-Att to get the refined text embedding for the query conditioned on each video, and compute text-video similarity scores. The videos are then ranked by these similarity scores to produce the final retrieval list.

4 Dataset Implementation Details

We use the **MSR-VTT** (Microsoft Research Video to Text) dataset, with 10,000 videos from 20 categories, annotated with 20 english sentences through crowd sourcing (Amazon Mechanical Turks) [7]. This dataset has a good spread of categories compared to other video datasets and provides 20 captions from different people to gather a deeper semantic understanding of a video (includes subjectivity of video content, different focuses of a video). However, as the paper we reference evaluates on 5 datasets and the other datasets only have 1 caption per video, we stick to using 1 randomly selected caption for each video to maintain consistency.

We only evaluated this pipeline with the dataset described instead of all five from the paper due to resource constraints (computation and storage, as video files are very large). Our code is at [wesleytruong/queries-are-not-alone](https://github.com/wesleytruong/queries-are-not-alone), and we use a single RTX 5090 for this experiment.

We downloaded the dataset from HuggingFace [friedrichor/MSR-VTT](https://huggingface.co/friedrichor/MSR-VTT), and used the openai-clip package to use the **ViT-B/16 CLIP** model. Openai’s implementation is at this page: [Clip implementation](https://openai.com/research/clip) The metrics used in our experiment are Recall at rank 1, 5, and 10 (R@1, R@5, R@10).

As most implementation details were not described in the paper, we relied on the pseudo-code and adapted it for PyTorch. One hyperparameter, K (K-nearest neighbors) aka how many of the nearest neighbors we use from a caption/query’s cluster for training and retrieval, was discussed in the paper. The authors determined that $K = 5$ has the highest retrieval accuracy across several datasets, and more than 10 neighbors introduced noise in the process. Thus, we use $K = 5$ in our experiments as well.

For pre-processing, we retrieve 8 frames that are evenly spaced within the video to use with CLIP.

As the text clusterer is trained separately from the rest of the components, we will separate the hyperparameters found for both in the tables below. Margin and dropout are part of the triplet-loss contrastive loss function: The loss penalizes cases where $D(a, p) + \alpha \geq D(a, n)$, where α is

the margin, D is cosine similarity, a is the anchor text, p is the positive text created via dropout on the anchor text, and n is a randomly sampled text known as negative. The loss function is $L_C = \max(0, D(a, p) - D(a, n) + \alpha)$. This is to push similar texts in the CLIP text embedding space closer together and push non-similar ones further apart. See Table 1 for exact values found.

In joint training (sweeper $\rightarrow$ vtc-attention $\rightarrow$ text-video fusion $\rightarrow$ retrieval), we use $K = 5$ as discussed above, 8 evenly spaced frames per video, and experiment with how many possible scores the Sweeper can assign through Jaccard similarity (these are treated as buckets, and are discussed more in the ablation). See Table 2 for exact values found.

In our ablation, we also experiment with some of these values.

Hyperparameter	Value
Total captions trained on	3000
Captions per vid (rand sel., pool of 20)	1
Batch size	8
Learning rate	1e-5
Margin (α)	0.2
Anchor dropout	0.1
Positive dropout	0.1
Epochs	1

Table 1: Text Clusterer hyper-parameters determined in our main experiment.

Hyperparameter	Value
Total captions trained on	9000
Captions per vid (rand sel., pool of 20)	1
Batch size	32
Learning rate	1e-4
Epochs	10
Jaccard similarity bins	4
K (K-NN)	5

Table 2: Joint training hyper-parameters determined in our main experiment.

5 Results

5.1 Retrieval Results

Our results were quite close to the paper’s results, but the paper’s results remained the most accurate for R@1, R@5, R@10 3. This is likely due to hyperparameter tuning that they may have done with their larger amount of compute, but our implementation tracks well with their results.

Method	R@1 $\uparrow$	R@5 $\uparrow$	R@10 $\uparrow$
X-Pool [1]	48.2	73.7	82.6
UATVR	50.8	76.3	85.5
CLIP-ViP [8]	54.2	77.2	84.8
VTC (Paper) [4]	54.8	79.2	88.7
VTC (Our reproduction)	52.8	75.8	83.1

Table 3: Text-to-video retrieval results on MSRVTT using CLIP-ViT-B/16.

Method	R@1 $\uparrow$	R@5 $\uparrow$	R@10 $\uparrow$
2 Buckets	51.1	74.0	81.0
4 Buckets	52.3	76.2	83.0
6 Buckets	52.5	76.8	83.1

Table 4: Ablation on number of buckets for Sweeper module scoring mechanism.

Method	R@1 $\uparrow$	R@5 $\uparrow$	R@10 $\uparrow$
0 captions	37.5	66.8	76.3
1000 captions	53.7	78.7	85.0
3000 captions	52.3	76.2	83.0
9000 captions	38.3	60.9	70.8

Table 5: Ablation on number of samples used to train the text clusterer

5.2 Exploration of number of sweeper buckets

One hyperparameter that was not explored in the paper was the number of labeling buckets used in the Jaccard similarity score loss function to train the sweeper module. Despite this being a potentially important hyperparameter, as the loss is used to provide semantic similarity signals, and the number of buckets may influence the complexity of knowledge that may be able to be captured by this loss function. In our experiment, we tested 3 different configurations of 2, 4, and 6 buckets respectively 4. We found that 6 buckets gave the best performance, supporting our hypothesis that a larger amount of buckets may capture more complex relationships within the text clusterer.

5.3 Exploration of training samples for clusterer

Another hyperparameter that is not explored in the paper is the training density for the text clusterer. Since the clusterer is trained in a separate loop from the joint training and only trains on text samples, there are multiple potential explorations for how we could train the text clusterer in order to potentially achieve better results. One such potential optimization could be to train the clusterer on a separate text-only dataset rather than sharing the same captions used in the joint training loop. Our exploration however focuses on varying the number of captios used to train the clusterer, to explore whether an upper limit or lower limit shows a specific behavior. In our results we found that a modest 1000/9000 text samples performed the best, whilst the full 9000 captions and 0 captions performed much more poorly 5.

Due to the poor result when using large number of captions, we suspect that expanding our dataset vertically by using a separate text corpus for training would not provide better performance. Some reasoning could be that since we are training for text-video retrieval with clip as the base model, training too many samples may overfit to the text dataset, but distort the embedding space too much, resulting in the further joint training being unable to realign the text and video embedding spaces for retrieval.

We additionally tested using 0 samples for training the cluster, i.e. using base clip text encoder as the clusterer model for finding K nearest neighbors. We thought this would be a fitting experiment, as in the Queries Are Not Alone paper, they perform ablation [4] on the text clusterer by sampling random text from the nearest neighbor candidate corpus. However, this doesn't precisely justify the training loop for the clusterer, as a potentially base Clip encoder may have sufficed without finetuning. However, after finding a decrease in $R@1$ of 30.1% as compared to using 1000 captions, we can confidently confirm that finetuning the clusterer is a necessary part of our architecture.

6 Our Contributions

Our primary contribution for this work is a faithful reproduction of the paper’s architecture while open-sourcing our work for reproducibility. Due to the high code quality, instructions, and sane modular configurations, our code can be easily extensible to run further experiments on other hyperparameters or datasets, simply by adjusting the configuration files. This may help with derivative works or more explorative dives into the paper’s architecture to gain keen insights such as performing different ablation studies such as we did.

References

- [1] Satya Krishna Gorti, Noel Vouitsis, Junwei Ma, Keyvan Golestan, Maksims Volkovs, Animesh Garg, and Guangwei Yu. X-pool: Cross-modal language-video attention for text-video retrieval, 2022. URL <https://arxiv.org/abs/2203.15086>.
- [2] Sarah Ibrahim, Xiaohang Sun, Pichao Wang, Amanmeet Garg, Ashutosh Sanan, and Mohamed Omar. Audio-enhanced text-to-video retrieval using text-conditioned feature alignment, 2023. URL <https://arxiv.org/abs/2307.12964>.
- [3] Jie Lei, Linjie Li, Luwei Zhou, Zhe Gan, Tamara L. Berg, Mohit Bansal, and Jingjing Liu. Less is more: Clipbert for video-and-language learning via sparse sampling, 2021. URL <https://arxiv.org/abs/2102.06183>.
- [4] Peiyang Liu, Xi Wang, Ziqiang Cui, and Wei Ye. Queries are not alone: Clustering text embeddings for video search. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR ’25, page 874–883, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400715921. doi: 10.1145/3726302.3730066. URL <https://doi.org/10.1145/3726302.3730066>.
- [5] Huaishao Luo, Lei Ji, Ming Zhong, Yang Chen, Wen Lei, Nan Duan, and Tianrui Li. Clip4clip: An empirical study of clip for end to end video clip retrieval, 2021. URL <https://arxiv.org/abs/2104.08860>.
- [6] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021. URL <https://arxiv.org/abs/2103.00020>.
- [7] Jun Xu, Tao Mei, Ting Yao, and Yong Rui. Msr-vtt: A large video description dataset for bridging video and language. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5288–5296, 2016. doi: 10.1109/CVPR.2016.571.
- [8] Hongwei Xue, Yuchong Sun, Bei Liu, Jianlong Fu, Ruihua Song, Houqiang Li, and Jiebo Luo. Clip-vip: Adapting pre-trained image-text model to video-language representation alignment, 2023. URL <https://arxiv.org/abs/2209.06430>.
- [9] Shuai Zhao, Linchao Zhu, Xiaohan Wang, and Yi Yang. Centerclip: Token clustering for efficient text-video retrieval. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR ’22, page 970–981. ACM, July 2022. doi: 10.1145/3477495.3531950. URL <http://dx.doi.org/10.1145/3477495.3531950>.